



LECTURE 11 — TIME COMPLEXITY (ADVANCED)

“Code likhna aasaan hai,
Code ka *growth behaviour* samajhna engineer banata hai.”



WHAT IS **TIME COMPLEXITY** (REAL MEANING)

Time Complexity ka matlab hota hai:

→ *Input size (n) badhne par program ke operations ka growth kaise ho raha hai*

- ✗ Runtime (seconds) nahi
 - ✗ Machine dependent nahi
 - ✓ Purely mathematical growth analysis
-

◆ EXAMPLE 1 — MULTIPLYING LOOP (×2)

```
for(int i = 1; i < n; i = i * 2) {  
    cout << "Hello Coder Army";  
}
```

Iteration Breakdown

Iteration	Value of i
1	1
2	2
3	4

4	8
...	...
k	2^k

Loop tab tak chalega jab:

$$2^k < n$$

Mathematical Derivation

$$2^k = n$$

$$\log(2^k) = \log(n)$$

$$k \log 2 = \log n$$

$$k = \log n / \log 2$$

$$k = \log_2 n$$

FINAL TIME COMPLEXITY

$$O(\log n)$$

Important Insight:

Jab bhi loop me **multiply ya divide** ho raha ho → **logarithmic growth**

◆ EXAMPLE 2 — MULTIPLYING LOOP (×3)

```
for(int i = 1; i < n; i = i * 3) {  
    cout << "Hello Coder Army";  
}
```

i values

1 → 3 → 9 → 27 → 81 → ...

Equation

$$3^k = n$$

$$k = \log_3 n$$

TIME COMPLEXITY

$$O(\log n)$$

Big-O Rule:

Log ka base **matter nahi karta**

$\log_2 n$, $\log_3 n$, $\log_{10} n \rightarrow$ sab $O(\log n)$

EXAMPLE 3 — DIVIDING WHILE LOOP

```
int i = n;
while(i != 0) {
    cout << "Hello";
    i = i / 2;
}
```

Flow

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

Math

$$n / 2^k = 1$$

$$2^k = n$$

$k = \log_2 n$

✅ TIME COMPLEXITY

$O(\log n)$

◆ EXAMPLE 4 — TRIPLE NESTED LOOP

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        for(int k = 0; k < n; k++) {  
            cout << "Hello";  
        }  
    }  
}
```

🔍 Execution Count

- Outer loop $\rightarrow n$
- Middle loop $\rightarrow n$
- Inner loop $\rightarrow n$

📊 Total Operations

$n \times n \times n = n^3$

✅ TIME COMPLEXITY

$O(n^3)$

📌 Rule:

◆ EXAMPLE 5 — SPECIAL DEPENDENT LOOP CASE

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < i * i; j++) {  
        cout << "Hello Coder Army";  
    }  
}
```

🔍 Key Observation

- Outer loop → n
- Inner loop → i^2 (not constant!)

📊 Total Work

$$\sum i^2 \quad (i = 0 \rightarrow n)$$
$$\approx n^3$$

✅ TIME COMPLEXITY

$$O(n^3)$$

⚠️ Why Special?

Inner loop ka size **i** ke saath grow kar raha hai
Isliye cube bana

●●● BEST / AVERAGE / WORST CASE ANALYSIS ●●●

🔍 Problem: Linear Search

```
int arr[] = {10, 20, 30, 40, 50};
```

```
int key = 30;
```

```
for(int i = 0; i < 5; i++) {  
    if(arr[i] == key) {  
        cout << "Found";  
        break;  
    }  
}
```

● BEST CASE

- Element **first position** pe mil jaye

Operations = 1

Time = $O(1)$

● WORST CASE

- Element **array me ho hi nahi**

Operations = n

Time = $O(n)$

AVERAGE CASE

- Element kahin bhi mil sakta hai
- Expected position $\approx n/2$

Operations $\approx n$

Time = $O(n)$

Conclusion:

Average case bhi **linear** hi hota hai

BIG-O, OMEGA, THETA — **TRUTH EXPOSED**

BIG-O (UPPER BOUND)

“Equal ya mujhse zyada”

Example:

Actual TC = n

Valid Big-O:

$O(n)$, $O(n^2)$, $O(n^3)$, $O(n^4)$...

✓ Sab definition ke according correct

👉 Isliye **Big-O** exact value nahi deta

OMEGA (LOWER BOUND)

“Equal ya mujhse kam”

Actual:

$$n^3$$

Valid:

$$\Omega(n^3), \Omega(n^2), \Omega(n), \Omega(1)$$

◆ THETA (EXACT)

Exact growth

$$\theta(n)$$

- ✗ Interviews me rarely
- ✗ Coding rounds me nahi
- ✓ Sirf theory books me



LINEAR TIME — TEXT GRAPH

Time ↑



➡ Input badhta hai → Time **linearly** badhta hai



MOST IMPORTANT REALITY

**Big-O ek safety limit hai,
exact measurement nahi.**

Isliye:

- Big-O = worst-case guarantee
 - Omega = useless for interviews
 - Theta = theoretical
-



INTERVIEW-PRO TIPS

- ✓ Multiply / Divide → $\log n$
 - ✓ Nested independent loops → multiply
 - ✓ Dependent loops → summation
 - ✓ Constants ignore karo
 - ✓ Logic samjho, formula nahi ratto
-



FINAL WRAP-UP (ONE GLANCE)

- Logarithmic loops → $O(\log n)$
- Nested loops → powers of n
- Average case $\approx$ Worst case
- Big-O $\neq$ exact
- Interviews → **Big-O only**

Made By Harshal Chauhan